

ЕТАП 7 ОТ 11 · РЪКОВОДСТВО

Кибла الأذان

Табът „Кибла“ вече е компас: стрелката сочи към Кааба, екранът казва накъде и с колко градуса да се завъртиш, а когато си точно в посоката, светва в злато и вибрира. Работи без интернет, на Android с и без Google услуги и на iPhone.

Компас

adhan Qibla()

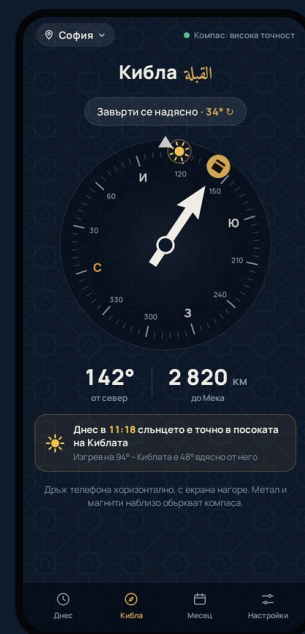
World Magnetic Model

expo-sensors

Слънцето

Jest – 149 теста

7 файла



Какво е решено в този етап

Етап 7 се прави преди етап 5 · посоката – по голямата окръжност (София 142°, 2 820 км) · Android: акселерометър + магнитометър, без разрешения и без Google · iPhone: компасът на iOS (иска разрешение за местоположение) · деклинация по WMM 2025 · „обърнат“ при $\pm 3^\circ$ + вибрация · калибриране при ниска точност · сензорите работят само докато екранът е отворен · слънцето на компаса и часът, в който е точно в посоката на Киблата.

СЪДЪРЖАНИЕ

01 Какво прави етапът

02 Нови пакети и нов build

03 Логика: посока, деклинация, компас

04 Компасът: useCompass

05 Екранът

06 Слънцето като проверка

07 Тестове

08 Проверка и край на етапа

Стъпка 1

Какво прави етапът

Две независими неща: **накъде е Киблата** (смята се от мястото, веднъж) и **накъде гледа телефонът** (компасът, непрекъснато). Разликата между тях е инструкцията „Завърти се надясно · 34°“.

Дърво на проекта



МЯСТО	КИБЛА ОТ СЕВЕР	ДО МЕКА
София	142°	2 820 км
Истанбул	152°	2 400 км
Единбург	120°	5 210 км
Ню Йорк	58°	10 300 км

Защо деклинацията е важна: компасът сочи магнитния север, а Киблата е спрямо истинския. В София разликата е ~5,7° – повече от допустимите ±3°. Затова тя се смята по World Magnetic Model (WMM 2025, валиден до 2029 г.) за точното място, без интернет.

Стъпка 2

Нови пакети и нов build

Три пакета; `expo-sensors` и `expo-haptics` имат native код – нужен е нов development build.

Инсталиране и нов build

```
npx expo install expo-sensors expo-haptics geomagnetism
Remove-Item -Recurse -Force android
npx expo run:android
```

- **expo-sensors** – акселерометър и магнитометър на Android. Не искат разрешения.
- **expo-haptics** – кратката вибрация, когато застанеш към Кибла.
- **geomagnetism** – World Magnetic Model на JavaScript (~70 KB): деклинация и сила на полето за всяко място.
- Plugin-ът на `expo-sensors` не се добавя в `app.json`: той иска разрешение за движение на iPhone, а там ползваме компаса на iOS.

Логика: посока, деклинация, компас

Чиста математика в `src/domain/qibla.ts` – без сензори, затова е покрита с тестове.

qibla.ts `src/domain`

- `qiblaBearing` – `Qibla()` от `adhan` : посоката по голямата окръжност (най-късият път по Земята).
- `turnInstruction` – най-късата посока на завъртане (и през $0^\circ/360^\circ$: от 350° до 10° е „надясно 20° “), „обърнат“ при $\pm 3^\circ$.
- `magneticInfo` – деклинация и очаквана сила на полето (μT) по WMM.
- `headingFromSensors` – посоката от акселерометъра и магнитометъра със същата формула като `SensorManager.getRotationMatrix` в Android: векторно произведение дава изток, второ – север; така наклонът на телефона не пречи.
- `smoothHeading` – изглаждане по окръжност, за да не прескача стрелката през 0° .
- `qualityFromField` – точност по силата на измереното поле спрямо очакваната: метал и магнити я отклоняват. `qualityFromAccuracy` – точността на iPhone в градуси.

`src/domain/qibla.ts`

129 реда

```
import { Coordinates, Qibla } from 'adhan';
import geomagnetism from 'geomagnetism';

import { distanceKm } from './geo';

/**
 * Кибла: посоката към Кааба и всичко около компаса – чиста математика, без сензори,
 * затова е покрито с тестове. Сензорите са в src/hooks/useCompass.ts.
 */

/** Кааба, Мека. */
export const KAABA = { latitude: 21.4225, longitude: 39.8262 };

/** До колко градуса разлика смятаме, че човек е обърнат към Кибла. */
export const ALIGN_DEG = 3;

/** Посоката към Кааба от север, по часовниковата стрелка (0–360°), по голямата окръжност. */
export function qiblaBearing(lat: number, lon: number): number {
  return Qibla(new Coordinates(lat, lon));
}

export function distanceToKaabaKm(lat: number, lon: number): number {
  return distanceKm(lat, lon, KAABA.latitude, KAABA.longitude);
}

/** 0–360 */
export function normalizeDeg(deg: number): number {
  return ((deg % 360) + 360) % 360;
}

/** Най-късата разлика от `from` до `to`: от -180 до 180 (положително = надясно). */
export function angleDelta(from: number, to: number): number {
  const d = normalizeDeg(to - from);
  return d > 180 ? d - 360 : d;
}
```

```

export interface TurnInstruction {
  /** Колко градуса да се завърти (винаги положително). */
  degrees: number;
  direction: 'left' | 'right';
  aligned: boolean;
}

/** Накъде и колко да се завърти човек, който гледа в `heading`, за да е към Кибла. */
export function turnInstruction(qibla: number, heading: number): TurnInstruction {
  const d = angleDelta(heading, qibla);
  return {
    degrees: Math.round(Math.abs(d)),
    direction: d >= 0 ? 'right' : 'left',
    aligned: Math.abs(d) <= ALIGN_DEG,
  };
}

/* ----- магнитно поле */

export interface MagneticInfo {
  /** Магнитна деклинация: колко градуса магнитният север е на изток от истинския. */
  declination: number;
  /** Очаквана сила на полето на това място (μT) – за проверка дали компасът е смутен. */
  fieldUT: number;
}

/**
 * Деклинация и сила на полето по World Magnetic Model (WMM 2025, валиден до 2029 г.).
 * В София ~+6°: без нея компасът би показал Киблата 6° встрани.
 */
export function magneticInfo(lat: number, lon: number, date = new Date()): MagneticInfo {
  const point = geomagnetism.model(date, { allowOutOfBoundsModel: true }).point([lat, lon]);
  return { declination: point.decl, fieldUT: point.f / 1000 };
}

/* ----- сензори (Android) */

type Vec = { x: number; y: number; z: number };

const cross = (a: Vec, b: Vec): Vec => ({
  x: a.y * b.z - a.z * b.y,
  y: a.z * b.x - a.x * b.z,
  z: a.x * b.y - a.y * b.x,
});

const norm = (a: Vec) => Math.hypot(a.x, a.y, a.z);

/**
 * Посока на горния край на телефона спрямо МАГНИТНИЯ север (0–360°) от акселерометъра
 * и магнитометъра – същата формула като SensorManager.getRotationMatrix + getOrientation
 * в Android, с поправка за наклона. null, ако данните не стигат (свободно падане,
 * телефонът е изправен вертикално).
 * Осите са тези на Android: x – надясно, y – към горния край, z – от екрана навън.
 */
export function headingFromSensors(gravity: Vec, magnetic: Vec): number | null {
  const H = cross(magnetic, gravity); // изток
  const hNorm = norm(H);
  const gNorm = norm(gravity);
  if (hNorm < 0.1 || gNorm < 0.1) return null;
  const h = { x: H.x / hNorm, y: H.y / hNorm, z: H.z / hNorm };
  const a = { x: gravity.x / gNorm, y: gravity.y / gNorm, z: gravity.z / gNorm };
  const m = cross(a, h); // север

```

```

    return normalizeDeg((Math.atan2(h.y, m.y) * 180) / Math.PI);
}

/**
 * Изглаждане на посоката (кръгово, за да не „прескача“ през 0°/360°).
 * `factor` 0-1: колко от новата стойност да се вземе.
 */
export function smoothHeading(prev: number | null, next: number, factor = 0.2): number {
  if (prev === null) return next;
  return normalizeDeg(prev + angleDelta(prev, next) * factor);
}

export type CompassQuality = 'high' | 'medium' | 'low';

/**
 * Точност на компаса по силата на измереното поле спрямо очакваната за мястото:
 * метал, магнити и некалибриран сензор я отклоняват.
 */
export function qualityFromField(measuredUT: number, expectedUT: number): CompassQuality {
  const ratio = measuredUT / expectedUT;
  if (ratio > 0.8 && ratio < 1.2) return 'high';
  if (ratio > 0.6 && ratio < 1.45) return 'medium';
  return 'low';
}

/** Точност на iPhone (градуси грешка от CoreLocation). Отрицателна = неизвестна. */
export function qualityFromAccuracy(deg: number): CompassQuality {
  if (deg < 0) return 'low';
  if (deg <= 15) return 'high';
  if (deg <= 30) return 'medium';
  return 'low';
}

```

Стъпка 4

Компасът: useCompass

Единственото място, което говори със сензорите.

useCompass.ts src/hooks

- **Android:** акселерометър и магнитометър на всеки 60 мс; нискочестотен филтър (остава само гравитацията, без треперенето на ръката), посока, деклинация, изглаждане. Без разрешения и без Google услуги – работи и на Huawei без GMS.
- **iPhone:** `Location.watchHeadingAsync` – компасът на iOS е калибриран от системата и дава точност в градуси. Изисква разрешение за местоположение; без него екранът показва бутон „Разреша местоположението“.
- `rotation` – „непрекъснатата“ посока (359° → 361°, а не → 1°), за да не се завърти скалата наобратно през 0°.
- Обновява екрана само при промяна над 0,4° – не 16 пъти в секунда.
- `active`: сензорите работят само докато табът „Кибла“ е отворен.

src/hooks/useCompass.ts

139 реда

```

import { Accelerometer, Magnetometer } from 'expo-sensors';
import * as Location from 'expo-location';

```

```

import { useEffect, useMemo, useState } from 'react';
import { Platform } from 'react-native';

import {
  angleDelta,
  headingFromSensors,
  magneticInfo,
  normalizeDeg,
  qualityFromAccuracy,
  qualityFromField,
  smoothHeading,
  type CompassQuality,
} from '@domain/qibla';

/**
 * Компасът на телефона.
 * - Android: акселерометър + магнитометър (expo-sensors). Не иска разрешения и не зависи
 * от Google услуги - работи и на Huawei без GMS. Деклинацията е от World Magnetic Model.
 * - iPhone: компасът на iOS (CoreLocation през expo-location) - калибриран от системата,
 * с точност в градуси. Изисква разрешение за местоположение.
 * Слуша сензорите само докато `active` е true (екранът „Кибла“ е отворен) - пести батерия.
 */

export type CompassStatus = 'starting' | 'ok' | 'unavailable' | 'needsPermission';

export interface CompassState {
  status: CompassStatus;
  /** Посока на горния край на телефона спрямо ИСТИНСКИЯ север, 0-360°. */
  heading: number | null;
  /**
   * Същата посока, но „непрекъсната“ (359° → 361°, не → 1°) - за плавно въртене
   * на компаса без завъртане наобратно през 0°.
   */
  rotation: number;
  quality: CompassQuality;
}

const INITIAL: CompassState = { status: 'starting', heading: null, rotation: 0, quality: 'high' };
const UPDATE_MS = 60;

export function useCompass(lat: number, lon: number, active: boolean): CompassState & { retry: () => void } {
  {
    const [state, setState] = useState<CompassState>(INITIAL);
    const [attempt, setAttempt] = useState(0);

    // деклинацията и очакваното поле се сменят бавно - закръгляме мястото до ~10 км
    const latKey = Math.round(lat * 10) / 10;
    const lonKey = Math.round(lon * 10) / 10;
    const info = useMemo(() => magneticInfo(latKey, lonKey), [latKey, lonKey]);

    useEffect(() => {
      if (!active) return;
      let alive = true;
      let heading: number | null = null;
      let rotation = 0;
      let lastSent = 0;
      let lastQuality: CompassQuality = 'high';
      const cleanups: (() => void)[] = [];

      const publish = (next: number, quality: CompassQuality) => {
        const prev = heading;

```

```

    heading = next;
    rotation += prev === null ? next : angleDelta(prev, next);
    const now = Date.now();
    // React не трябва да се прерисува 16 пъти в секунда за промени под половин градус
    if (prev !== null && Math.abs(angleDelta(prev, next)) < 0.4 && quality === lastQuality && now -
lastSent < 400) {
        return;
    }
    lastSent = now;
    lastQuality = quality;
    if (alive) setState({ status: 'ok', heading: next, rotation, quality });
};

(async () => {
    if (Platform.OS === 'android') {
        const [hasMag, hasAcc] = await Promise.all([Magnetometer.isAvailableAsync(),
Accelerometer.isAvailableAsync()]);
        if (!alive) return;
        if (!hasMag || !hasAcc) {
            setState({ ...INITIAL, status: 'unavailable' });
            return;
        }
        let gravity: { x: number; y: number; z: number } | null = null;
        let field: { x: number; y: number; z: number } | null = null;
        let fieldRatio = 1;
        Accelerometer.setUpdateInterval(UPDATE_MS);
        Magnetometer.setUpdateInterval(UPDATE_MS);
        const acc = Accelerometer.addListener((a) => {
            // лек нискочестотен филтър: остава само гравитацията, без треперенето на ръката
            gravity = gravity
                ? { x: gravity.x * 0.8 + a.x * 0.2, y: gravity.y * 0.8 + a.y * 0.2, z: gravity.z * 0.8 + a.z *
0.2 }
                : a;
        });
        const mag = Magnetometer.addListener((m) => {
            field = field
                ? { x: field.x * 0.7 + m.x * 0.3, y: field.y * 0.7 + m.y * 0.3, z: field.z * 0.7 + m.z * 0.3 }
                : m;
            if (!gravity) return;
            const magnetic = headingFromSensors(gravity, field);
            if (magnetic === null) return;
            fieldRatio = fieldRatio * 0.9 + (Math.hypot(m.x, m.y, m.z) / info.fieldUT) * 0.1;
            const trueHeading = normalizeDeg(magnetic + info.declination);
            publish(smoothHeading(heading, trueHeading, 0.25), qualityFromField(fieldRatio * info.fieldUT,
info.fieldUT));
        });
        cleanups.push(() => acc.remove(), () => mag.remove());
        return;
    }

    if (Platform.OS === 'ios') {
        const perm = await Location.getForegroundPermissionsAsync();
        if (!alive) return;
        if (perm.status !== 'granted') {
            setState({ ...INITIAL, status: 'needsPermission' });
            return;
        }
        try {
            const sub = await Location.watchHeadingAsync((h) => {
                const trueHeading = h.trueHeading >= 0 ? h.trueHeading : normalizeDeg(h.magHeading +
info.declination);

```

```

        publish(smoothHeading(heading, trueHeading, 0.5), qualityFromAccuracy(h.accuracy));
    });
    if (!alive) sub.remove();
    else cleanups.push(() => sub.remove());
  } catch {
    if (alive) setState({ ...INITIAL, status: 'unavailable' });
  }
  return;
}

setState({ ...INITIAL, status: 'unavailable' }); // ye6
})();

return () => {
  alive = false;
  cleanups.forEach((c) => c());
};
}, [active, info, attempt]);

return { ...state, retry: () => setAttempt((a) => a + 1) };
}

```

Стъпка 5

Екранът

Точно по одобрения mockup.

CompassDial.tsx src/components/qibla

- Четири слоя: фон с рамка и сияние (злато при „обърнат“); скалата, която се върти (чертички, „С/И/Ю/З“, градуси, знакът на Кааба); стрелката към Кааба; триъгълникът за посоката на телефона.
- Буквите и числата се въртят обратно на скалата – остават изправени.
- Въртенето е с Reanimated (120 мс) – плавно и без да натоварва JavaScript.

src/components/qibla/CompassDial.tsx

194 реда

```

import { useEffect } from 'react';
import { StyleSheet, Text, View } from 'react-native';
import Animated, { useAnimatedStyle, useSharedValue, withTiming } from 'react-native-reanimated';
import Svg, { Circle, Defs, G, Line, Path, RadialGradient, Rect, Stop } from 'react-native-svg';

import { angleDelta } from '@/domain/qibla';
import { colors } from '@/theme/colors';
import { fonts } from '@/theme/typography';

interface Props {
  size: number;
  /** Посока към Кааба от север (0-360). */
  qibla: number;
  /** Непрекъсната посока на телефона (виж useCompass → rotation). */
  rotation: number;
  aligned: boolean;
  /** Къде е слънцето (0-360° от север), когато е над хоризонта; иначе null. */
  sun?: number | null;
  /** С, И, Ю, З / N, E, S, W */
  letters: readonly string[];
  accessibilityLabel: string;
}

```

```

}

const GOLD = colors.gold;
const SUN = colors.sun;
const NUMBERS = [30, 60, 120, 150, 210, 240, 300, 330];
/** Въртящият се слой е малко по-голям от компаса, за да не се отрязва сиянието на слънцето. */
const PAD = 12;
/** Слънцето и Кааба са „заедно“ – слънцето огрява знака на Кааба. */
const SUN_WITH_KAABA_DEG = 3;

/**
 * Компасът: скалата се върти така, че „С“ винаги сочи истинския север; знакът на Кааба
 * е на ръба в посоката на Киблата; стрелката в средата сочи към Кааба спрямо телефона;
 * триъгълникът горе показва накъде гледа телефонът. Буквите и числата остават изправени.
 */
export function CompassDial({ size, qibla, rotation, aligned, sun = null, letters, accessibilityLabel }:
Props) {
  const rot = useSharedValue(rotation);
  useEffect(() => {
    rot.set(withTiming(rotation, { duration: 120 }));
  }, [rotation, rot]);

  const dialStyle = useAnimatedStyle(() => ({ transform: [{ rotate: `${-rot.get()}deg` }] }));
  const uprightStyle = useAnimatedStyle(() => ({ transform: [{ rotate: `${rot.get()}deg` }] }));
  const arrowStyle = useAnimatedStyle(() => ({ transform: [{ rotate: `${qibla - rot.get()}deg` }] }));

  const c = size / 2;
  const R = c - 22; // радиус на скалата
  const ring = aligned ? GOLD : 'rgba(255,255,255,0.10)';
  const arrow = aligned ? GOLD : colors.text;

  // позиция на етикет на ъгъл `deg` и разстояние `r` от центъра
  const at = (deg: number, r: number) => {
    const a = (deg * Math.PI) / 180;
    return { left: c + r * Math.sin(a), top: c - r * Math.cos(a) };
  };

  const ticks = [];
  for (let d = 0; d < 360; d += 5) {
    const long = d % 30 === 0;
    const a = (d * Math.PI) / 180;
    const r1 = R - (long ? 14 : 7);
    ticks.push(
      <Line
        key={d}
        x1={c + r1 * Math.sin(a)}
        y1={c - r1 * Math.cos(a)}
        x2={c + R * Math.sin(a)}
        y2={c - R * Math.cos(a)}
        stroke={`rgba(242,239,232,${long ? 0.55 : 0.22})`}
        strokeWidth={long ? 2 : 1.2}
        strokeLinecap="round"
      />,
    );
  }

  const kaaba = at(qibla, R + 2);
  const sunWithKaaba = sun !== null && Math.abs(angleDelta(qibla, sun)) <= SUN_WITH_KAABA_DEG;
  const sunPos = sun !== null && !sunWithKaaba ? at(sun, R + 2) : null;

  return (

```

```

<View
  style={{ width: size, height: size }}
  accessible
  accessibilityRole="image"
  accessibilityLabel={accessibilityLabel}
>
  /* фон, рамка и сияние (не се върти) */
  <Svg width={size} height={size} style={StyleSheet.absoluteFill}>
    <Defs>
      <RadialGradient id="glow" cx="50%" cy="50%" r="50%">
        <Stop offset="0" stopColor={aligned ? 'rgba(212,168,87,0.35)' : 'rgba(255,255,255,0.06)'} />
        <Stop offset="1" stopColor="rgba(0,0,0,0)" />
      </RadialGradient>
    </Defs>
    <Circle cx={c} cy={c} r={c - 6} fill="rgba(6,10,20,0.45)" stroke={ring} strokeWidth={aligned ? 2.5
: 1} />
    <Circle cx={c} cy={c} r={R - 14} fill="url(#glow)" />
  </Svg>

  /* скалата - върти се */
  <Animated.View style={[styles.dial, { width: size + 2 * PAD, height: size + 2 * PAD }, dialStyle]}>
    <Svg width={size + 2 * PAD} height={size + 2 * PAD} viewBox={`$${-PAD} $${-PAD} ${size + 2 * PAD}
${size + 2 * PAD}`}>
      {ticks}
      {sunPos && <SunMark x={sunPos.left} y={sunPos.top} />}
      {sunWithKaaba && (
        <G>
          <Circle cx={kaaba.left} cy={kaaba.top} r={27} fill="rgba(246,196,83,0.18)" />
          <SunRays x={kaaba.left} y={kaaba.top} r1={20} r2={26} />
        </G>
      )}
      <G transform={`translate(${kaaba.left} ${kaaba.top}) rotate(${qibla})`}>
        <Circle r={16} fill={GOLD} />
        <Rect x={-7} y={-7.5} width={14} height={15} rx={1.5} fill={colors.goldInk} />
        <Rect x={-7} y={-4.2} width={14} height={2.4} fill={GOLD} />
      </G>
    </Svg>
    {letters.map((l, i) => {
      const p = at(i * 90, R - 30);
      return (
        <Animated.View key={l} style={[styles.label, { left: PAD + p.left - 14, top: PAD + p.top - 12
}, uprightStyle]}>
          <Text style={styles.letter, i === 0 && styles.north}>{l}</Text>
        </Animated.View>
      );
    })}
    {NUMBERS.map((n) => {
      const p = at(n, R - 28);
      return (
        <Animated.View key={n} style={[styles.label, { left: PAD + p.left - 14, top: PAD + p.top - 12
}, uprightStyle]}>
          <Text style={styles.number}>{n}</Text>
        </Animated.View>
      );
    })}
  </Animated.View>

  /* стрелката към Кааба */
  <Animated.View style={[StyleSheet.absoluteFill, arrowStyle]}>
    <Svg width={size} height={size}>
      <Path

```

```

        d={`M${c} ${c - R + 30} L${c + 18} ${c - R + 74} L${c + 6} ${c - R + 70} L${c + 6} ${c + 32}
L${c - 6} ${c + 32} L${c - 6} ${c - R + 70} L${c - 18} ${c - R + 74} Z`}
        fill={arrow}
      />
      <Circle cx={c} cy={c} r={11} fill={colors.base} stroke={arrow} strokeWidth={3} />
    </Svg>
  </Animated.View>

  /* накъде гледа телефонът - винаги горе */
  <Svg width={size} height={size} style={StyleSheet.absoluteFill} pointerEvents="none">
    <Path d={`M${c} 0 L${c + 10} 16 L${c - 10} 16 Z`} fill={aligned ? GOLD : 'rgba(242,239,232,0.85)'} />
  />
  </Svg>
</View>
);
}

/** Лъчите на слънцето около точка. */
function SunRays({ x, y, r1, r2 }: { x: number; y: number; r1: number; r2: number }) {
  return (
    <G>
      {[0, 45, 90, 135, 180, 225, 270, 315].map((d) => {
        const a = (d * Math.PI) / 180;
        return (
          <Line
            key={d}
            x1={x + r1 * Math.sin(a)}
            y1={y - r1 * Math.cos(a)}
            x2={x + r2 * Math.sin(a)}
            y2={y - r2 * Math.cos(a)}
            stroke={SUN}
            strokeWidth={2.2}
            strokeLinecap="round"
          />
        );
      })}
    </G>
  );
}

/** Знакът на слънцето върху ръба на компаса. */
function SunMark({ x, y }: { x: number; y: number }) {
  return (
    <G>
      <Circle cx={x} cy={y} r={16} fill={colors.base} stroke="rgba(246,196,83,0.55)" strokeWidth={1} />
      <SunRays x={x} y={y} r1={9.5} r2={13} />
      <Circle cx={x} cy={y} r={6.5} fill={SUN} />
    </G>
  );
}

const styles = StyleSheet.create({
  dial: { position: 'absolute', left: -PAD, top: -PAD },
  label: { position: 'absolute', width: 28, height: 24, alignItems: 'center', justifyContent: 'center' },
  letter: { fontFamily: fonts.extrabold, fontSize: 16, color: 'rgba(242,239,232,0.8)', includeFontPadding: false },
  north: { color: colors.warn },
  number: { fontFamily: fonts.semibold, fontSize: 10, color: colors.muted, includeFontPadding: false },
});

```

qibla.tsx src/app/(tabs)

- Горѐ: мястото (тап – изборът на място) и точността на компаса.
- Лента: „Компасът се включва...“ / „Завърти се надясно · 34° ъ“ / „✓ Обърнат си към Кибла“ (злато) + кратка вибрация.
- Градуси от север и разстояние до Мека (2 820 км – без Intl).
- Карти: „Калибрирай компаса“ (осмица) при ниска точност; „Няма компас“ (телефон без магнитен сензор) – неподвижна скала и обяснение с градусите; „Нужно е разрешение“ (iPhone).
- `useIsFocused` пуска сензорите само докато екранът е отворен.

src/app/(tabs)/qibla.tsx

367 реда

```
import * as Haptics from 'expo-haptics';
import * as Location from 'expo-location';
import { router, useIsFocused } from 'expo-router';
import { useEffect, useRef } from 'react';
import { ActivityIndicator, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import Svg, { Circle, Line, Path, Rect } from 'react-native-svg';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { ChevronDownIcon, PinIcon } from '@components/icons';
import { CompassDial } from '@components/qibla/CompassDial';
import { useTabBarHeight } from '@components/TabBar';
import { formatHM } from '@domain/format';
import { distanceToKaabaKm, qiblaBearing, turnInstruction } from '@domain/qibla';
import { isSunUp, sunPosition, sunQiblaHint, sunriseBearing, type SunQiblaHint } from '@domain/sun';
import { useCompass } from '@hooks/useCompass';
import { useNow } from '@hooks/useNow';
import { useI18n } from '@i18n';
import { selectLocation, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

const QUALITY_COLOR = { high: '#5FBF8F', medium: colors.gold, low: colors.warn } as const;

/** Екран „Кибла“: компас със стрелка към Кааба, градуси от север и разстояние до Мека. */
export default function QiblaScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t } = useI18n();
  const q = t.qibla;
  const location = useSettings(selectLocation);
  const hasPlace = useSettings((s) => s.gpsLocation !== null || s.manualLocation !== null);

  // сензорите работят само докато екранът е отворен – пести батерия
  const focused = useIsFocused();
  const compass = useCompass(location.latitude, location.longitude, focused);

  const bearing = qiblaBearing(location.latitude, location.longitude);
  const bearingText = Math.round(bearing);
  const km = Math.round(distanceToKaabaKm(location.latitude, location.longitude));
  // „2 820“ – с интервал за хилядите (неразделим); без Intl (в Hermes е бавен и различен по телефони)
  const kmText = String(km).replace(/B(?=(\d{3})+(?!\d))/g, '\u00A0');

  // слънцето: къде е сега и кога е точно в посоката на Киблата (обновява се на 30 сек.;
  // React Compiler пази резултатите между прерисуванията от компаса)
  const now = useNow(30_000);
```

```

const sunNow = sunPosition(now, location.latitude, location.longitude);
const sunAzimuth = isSunUp(sunNow) ? sunNow.azimuth : null;
const hint = sunQiblaHint(now, location.latitude, location.longitude, bearing);
const sunriseText = hint ? sunriseLine(hint, location.latitude, location.longitude, bearing, q.sunrise) :
null;

const live = compass.status === 'ok' && compass.heading !== null;
const turn = live ? turnInstruction(bearing, compass.heading!) : null;
const aligned = !!turn?.aligned;

// кратка вибрация, когато човек застане точно към Кибла
const wasAligned = useRef(false);
useEffect(() => {
  if (aligned && !wasAligned.current) {
    Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success).catch(() => {});
  }
  wasAligned.current = aligned;
}, [aligned]);

const allowLocation = async () => {
  await Location.requestForegroundPermissionsAsync();
  compass.retry();
};

// без компас скалата стои неподвижно със „C“ горе, а стрелката сочи Киблата от север
const rotation = live ? compass.rotation : 0;

return (
  <View style={styles.root}>
    <GeometricPattern />
    <ScrollView
      style={{ marginBottom: tabBarHeight }}
      contentContainerStyle={[styles.content, { paddingTop: insets.top + 8 }]}
      showsVerticalScrollIndicator={false}
    >
      <View style={styles.top}>
        <Pressable
          onPress={() => router.push('/place')}
          accessibilityRole="button"
          accessibilityLabel={t.ally.city(location.names[lang])}
          style={({ pressed }) => [styles.city, pressed && styles.pressed]}
        >
          <PinIcon color={colors.text} />
          <Text style={styles.cityText}>{hasPlace ? location.names[lang] : t.choosePlace}</Text>
          <ChevronDownIcon color={colors.text} />
        </Pressable>
        {live && (
          <View style={styles.acc}>
            <View style={[styles.dot, { backgroundColor: QUALITY_COLOR[compass.quality] }]} />
            <Text style={styles.accText}>{q.accuracy(q.quality[compass.quality])}</Text>
          </View>
        )}
      </View>

      <View style={styles.titleRow}>
        <Text style={styles.title}>{q.title}</Text>
        <Text style={styles.arabic}>القبلة</Text>
      </View>

      <View style={[styles.pill, aligned && styles.pillGold]}>
        {compass.status === 'starting' ? (

```

```

        <>
            <ActivityIndicator size="small" color={colors.gold} />
            <Text style={styles.pillText}>{q.starting}</Text>
        </>
    ) : aligned ? (
        <Text style={[styles.pillText, styles.pillTextGold]}>{q.aligned}</Text>
    ) : turn ? (
        <Text style={styles.pillText}>
            {q.turnLabel(turn.direction)} ·{' '}
        <Text style={styles.pillDeg}>
            {turn.degrees}° {turn.direction === 'right' ? '↻' : '↺'}
        </Text>
    </Text>
    ) : (
        <Text style={styles.pillText}>{compass.status === 'needsPermission' ? q.permissionTitle :
q.noCompassTitle}</Text>
    )}
</View>

<View style={styles.dial}>
    <CompassDial
        size={300}
        qibla={bearing}
        rotation={rotation}
        aligned={aligned}
        sun={sunAzimuth}
        letters={q.letters}
        accessibilityLabel={q.a11yDial(bearingText)}
    />
</View>

<View style={styles.nums}>
    <View>
        <Text style={styles.big}>{bearingText}°</Text>
        <Text style={styles.lbl}>{q.fromNorth}</Text>
    </View>
    <View style={styles.sep} />
    <View>
        <Text style={styles.big}>
            {kmText} <Text style={styles.unit}>{q.km}</Text>
        </Text>
        <Text style={styles.lbl}>{q.toMecca}</Text>
    </View>
</View>

{hint && <SunCard hint={hint} sub={sunriseText} />}

{compass.status === 'unavailable' ? (
    <Card title={q.noCompassTitle} text={q.noCompassText(bearingText)} />
) : compass.status === 'needsPermission' ? (
    <Card title={q.permissionTitle} text={q.permissionText} action={{ label: q.allowLocation,
onPress: allowLocation }} />
) : live && compass.quality === 'low' ? (
    <Card title={q.calibrateTitle} text={q.calibrateText} figureEight />
) : (
    <Text style={styles.tip}>{q.tip}</Text>
)}
</ScrollView>
</View>
);
}

```

```

/** „Изгрев на 94° – Киблата е 48° вдясно от него” – за деня от подсказката. */
function sunriseLine(
  hint: SunQiblaHint,
  lat: number,
  lon: number,
  qibla: number,
  format: (azimuth: number, degrees: number, side: 'left' | 'right' | 'same') => string,
): string | null {
  const b = sunriseBearing(hint.time, lat, lon, qibla);
  return b ? format(Math.round(b.azimuth), b.degrees, b.side) : null;
}

/** „Днес в 11:18 слънцето е точно в посоката на Киблата” + изгревът спрямо Киблата. */
function SunCard({ hint, sub }: { hint: SunQiblaHint; sub: string | null }) {
  const { t } = useI18n();
  const q = t.qibla;
  const gold = hint.kind === 'now';
  const template = hint.kind === 'now' ? q.sunNow : hint.kind === 'today' ? q.sunToday : q.sunTomorrow;
  const [before, after = ''] = template.split('{t}');
  const hasTime = template.includes('{t}');
  return (
    <View style={[styles.sunCard, gold && styles.sunCardGold]} accessible>
      <SunIcon color={gold ? colors.goldInk : colors.sun} />
      <View style={styles.cardText}>
        <Text style={[styles.sunMain, gold && styles.sunMainGold]}>
          {before}
          {hasTime && <Text style={styles.sunTime, gold && styles.sunMainGold}>{formatHM(hint.time)}}
        </Text>
        {after}
      </Text>
      {sub && !gold && <Text style={styles.sunSub}>{sub}</Text>}
    </View>
  </View>
);
}

function SunIcon({ color }: { color: string }) {
  const rays = [0, 45, 90, 135, 180, 225, 270, 315];
  return (
    <Svg width={26} height={26} viewBox="-13 -13 26 26">
      {rays.map((d) => {
        const a = (d * Math.PI) / 180;
        return (
          <Line
            key={d}
            x1={8 * Math.sin(a)}
            y1={-8 * Math.cos(a)}
            x2={11.5 * Math.sin(a)}
            y2={-11.5 * Math.cos(a)}
            stroke={color}
            strokeWidth={2.2}
            strokeLinecap="round"
          />
        );
      })}
      <Circle r={5.5} fill={color} />
    </Svg>
  );
}

```

```

function Card({
  title,
  text,
  action,
  figureEight,
}: {
  title: string;
  text: string;
  action?: { label: string; onPress: () => void };
  figureEight?: boolean;
}) {
  return (
    <View style={styles.card}>
      {figureEight && (
        <Svg width={64} height={40} viewBox="0 0 74 44">
          <Path
            d="M37 22 C 50 4, 70 4, 70 22 C 70 40, 50 40, 37 22 C 24 4, 4 4, 4 22 C 4 40, 24 40, 37 22 Z"
            fill="none"
            stroke={colors.warn}
            strokeWidth={2.5}
            strokeDasharray="5 4"
          />
          <Rect x={31} y={14} width={12} height={17} rx={3} fill={colors.text} />
        </Svg>
      )}
      <View style={styles.cardText}>
        <Text style={styles.cardTitle}>{title}</Text>
        <Text style={styles.cardBody}>{text}</Text>
        {action && (
          <Pressable onPress={action.onPress} accessibilityRole="button" style={styles.cardBtn}>
            <Text style={styles.cardBtnText}>{action.label}</Text>
          </Pressable>
        )}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  content: { alignItems: 'center', paddingBottom: 24 },
  top: {
    alignSelf: 'stretch',
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    paddingHorizontal: 18,
  },
  city: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
    paddingVertical: 7,
    paddingLeft: 10,
    paddingRight: 12,
    borderRadius: 999,
    backgroundColor: colors.pill,
  },
  pressed: { opacity: 0.7 },
  cityText: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },
  acc: { flexDirection: 'row', alignItems: 'center', gap: 6, flexShrink: 1 },

```

```

dot: { width: 8, height: 8, borderRadius: 4 },
accText: { fontFamily: fonts.semibold, fontSize: 11.5, color: colors.muted },
titleRow: { flexDirection: 'row', alignItems: 'baseline', gap: 10, marginTop: 16 },
title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text },
arabic: { fontFamily: fonts.arabicBold, fontSize: 24, color: colors.gold },
pill: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 8,
  marginTop: 10,
  paddingHorizontal: 16,
  paddingVertical: 9,
  borderRadius: 999,
  backgroundColor: 'rgba(255,255,255,0.08)',
  borderWidth: 1,
  borderColor: 'rgba(255,255,255,0.12)',
},
pillGold: { backgroundColor: colors.gold, borderColor: colors.gold },
pillText: { fontFamily: fonts.semibold, fontSize: 14, color: colors.text },
pillTextGold: { fontFamily: fonts.extrabold, color: colors.goldInk },
pillDeg: { fontFamily: fonts.extrabold, color: colors.gold, ...tabularNums },
dial: { marginTop: 14 },
nums: { flexDirection: 'row', alignItems: 'center', gap: 22, marginTop: 10 },
big: { fontFamily: fonts.extrabold, fontSize: 30, color: colors.text, textAlign: 'center', ...tabularNums },
},
unit: { fontFamily: fonts.bold, fontSize: 16, color: colors.muted },
lbl: { fontFamily: fonts.semibold, fontSize: 12, color: colors.muted, textAlign: 'center' },
sep: { width: StyleSheet.hairlineWidth, height: 40, backgroundColor: 'rgba(255,255,255,0.2)' },
tip: {
  marginTop: 16,
  marginHorizontal: 28,
  fontFamily: fonts.regular,
  fontSize: 12.5,
  lineHeight: 18,
  color: colors.muted,
  textAlign: 'center',
},
sunCard: {
  alignSelf: 'stretch',
  marginTop: 14,
  marginHorizontal: 16,
  flexDirection: 'row',
  alignItems: 'center',
  gap: 12,
  paddingVertical: 11,
  paddingHorizontal: 14,
  borderRadius: 16,
  backgroundColor: 'rgba(246,196,83,0.08)',
  borderWidth: 1,
  borderColor: 'rgba(246,196,83,0.28)',
},
sunCardGold: { backgroundColor: colors.gold, borderColor: colors.gold },
sunMain: { fontFamily: fonts.bold, fontSize: 13.5, lineHeight: 18, color: colors.text },
sunMainGold: { color: colors.goldInk },
sunTime: { fontFamily: fonts.extrabold, color: colors.sun, ...tabularNums },
sunSub: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 16, color: colors.muted },
card: {
  alignSelf: 'stretch',
  marginTop: 14,
  marginHorizontal: 16,
  flexDirection: 'row',

```

```

    alignItems: 'center',
    gap: 12,
    padding: 14,
    borderRadius: 16,
    backgroundColor: 'rgba(227,154,75,0.14)',
    borderWidth: 1,
    borderColor: 'rgba(227,154,75,0.45)',
  },
  cardText: { flex: 1, gap: 3 },
  cardTitle: { fontFamily: fonts.extrabold, fontSize: 14, color: colors.warn },
  cardBody: { fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.text },
  cardBtn: {
    alignSelf: 'flex-start',
    marginTop: 6,
    paddingHorizontal: 12,
    paddingVertical: 6,
    borderRadius: 999,
    backgroundColor: colors.gold,
  },
  cardBtnText: { fontFamily: fonts.bold, fontSize: 12.5, color: colors.goldInk },
});

```

src/i18n/strings.ts – текстовете (и на английски)

```

qibla: {
  title: 'Кибла',
  aligned: '✓ Обърнат си към Кибла',
  turnLabel: (direction) => `Завърти се ${direction === 'right' ? 'надясно' : 'наляво'}`,
  accuracy: (q) => `Компас: ${q} точност`,
  calibrateTitle: 'Калибрирай компаса',
  noCompassText: (deg) => `Този телефон няма магнитен сензор. Киблата е на ${deg}° от север: ...`,
  letters: ['C', 'И', 'Ю', 'З'], // на английски N, E, S, W
}

```

Стъпка 6

Слънцето като проверка

Компасът на телефона греши с 5–10°, а слънцето – никога. Затова на компаса има ✱ там, където е слънцето в момента, а под числата – в колко часа днес то е точно в посоката на Киблата. В този момент може да се обърнеш с лице към слънцето и си към Кибла – без компас.

ЧАС (СОФИЯ, 1 ОКТ.)	КОМПАСЪТ	КАРТАТА
09:30	✱ на 116°, до знака на Кааба	„Днес в 11:18 слънцето е точно в посоката на Киблата“ + „Изгрев на 94° – Киблата е 48° вдясно от него“
11:18 (±5 мин)	слънцето огрява знака на Кааба	златна: „Сега слънцето е точно в посоката на Киблата“
20:30	✱ го няма – залязло е	„Утре в 11:17 ...“

sun.ts src/domain

- **sunPosition** – посока и височина на слънцето за всеки момент и място (формулите на NOAA, точност ~0,1°, без интернет).

- `sunAtAzimuth` – кога в деня слънцето е в дадена посока: стъпки по 5 минути и после делене на интервала наполовина, закръглено до минута. Само ако е поне 5° над хоризонта – по-ниско е зад сгради и не помага.
- `sunQiblaHint` – „сега“ (±5 мин), „днес в ...“ или „утре в ...“; `null`, ако слънцето не минава оттам (напр. Ню Йорк през октомври, полярна нощ) – тогава картата не се показва.
- `sunriseBearing` – къде изгрява слънцето спрямо Киблата. Смята истинския изгрев (07:24 в София), а не часа „Изгрев“ от календара на Мюфтийството (07:16) – той е по-рано нарочно, като предпазен интервал за края на сутрешната молитва. Затова в картата има само посоката, без час – за да не се разминава с екрана „Днес“.

src/domain/sun.ts

160 реда

```
import { angleDelta, normalizeDeg } from './qibla';

/**
 * Слънцето спрямо Киблата: къде е слънцето в момента и в колко часа днес е точно
 * в посоката на Кааба. Изчислява се на телефона, без интернет
 * (формулите на NOAA / Meeus с ниска точност, около 0,1° – с много запас за компас).
 */

const RAD = Math.PI / 180;
const DAY_MS = 86_400_000;
const J2000 = Date.UTC(2000, 0, 1, 12);

/** Под тази височина слънцето е зад хоризонта (с пречупването в атмосферата). */
export const SUN_HORIZON = -0.833;
/** Слънце по-ниско от това е зад сгради и дървета – не е полезен ориентир. */
export const SUN_MIN_ALTITUDE = 5;
/** „Сега“: до 5 минути преди или след точния момент. */
export const SUN_NOW_MS = 5 * 60_000;

export interface SunPosition {
  /** От север, по часовниковата стрелка, 0–360°. */
  azimuth: number;
  /** Височина над хоризонта в градуси (отрицателна = под хоризонта). */
  altitude: number;
}

export function sunPosition(date: Date, lat: number, lon: number): SunPosition {
  const n = (date.getTime() - J2000) / DAY_MS;
  const meanLon = normalizeDeg(280.46 + 0.9856474 * n);
  const g = normalizeDeg(357.528 + 0.9856003 * n) * RAD;
  const lambda = (meanLon + 1.915 * Math.sin(g) + 0.02 * Math.sin(2 * g)) * RAD;
  const eps = (23.439 - 0.0000004 * n) * RAD;

  const ra = Math.atan2(Math.cos(eps) * Math.sin(lambda), Math.cos(lambda));
  const dec = Math.asin(Math.sin(eps) * Math.sin(lambda));
  const gmst = normalizeDeg(280.46061837 + 360.98564736629 * n);
  const ha = (gmst + lon) * RAD - ra;
  const phi = lat * RAD;

  const altitude = Math.asin(Math.sin(phi) * Math.sin(dec) + Math.cos(phi) * Math.cos(dec) * Math.cos(ha));
  const azimuth = Math.atan2(-Math.sin(ha), Math.tan(dec) * Math.cos(phi) - Math.sin(phi) * Math.cos(ha));
  return { azimuth: normalizeDeg(azimuth / RAD), altitude: altitude / RAD };
}

export const isSunUp = (p: SunPosition) => p.altitude > SUN_HORIZON;
```

```

/**
 * Кога в този ден (местно време на телефона) слънцето е точно в посоката `azimuth`
 * и поне на `minAltitude` над хоризонта. null, ако не минава оттам
 * (например далеч на север през зимата или когато е твърде ниско).
 */
export function sunAtAzimuth(
  day: Date,
  lat: number,
  lon: number,
  azimuth: number,
  minAltitude = SUN_MIN_ALTITUDE,
): Date | null {
  const start = new Date(day.getFullYear(), day.getMonth(), day.getDate()).getTime();
  const end = new Date(day.getFullYear(), day.getMonth(), day.getDate() + 1).getTime();
  const STEP = 5 * 60_000;
  const diff = (t: number) => angleDelta(azimuth, sunPosition(new Date(t), lat, lon).azimuth);

  let prevT = start;
  let prev = diff(start);
  for (let t = start + STEP; t <= end; t += STEP) {
    const cur = diff(t);
    // смяна на знака близо до посоката (а не скокът ±180° от другата страна)
    if (Math.sign(cur) !== Math.sign(prev) && Math.abs(cur) < 45 && Math.abs(prev) < 45) {
      let lo = prevT;
      let hi = t;
      let loVal = prev;
      for (let i = 0; i < 20; i++) {
        const mid = (lo + hi) / 2;
        const v = diff(mid);
        if (Math.sign(v) === Math.sign(loVal)) {
          lo = mid;
          loVal = v;
        } else {
          hi = mid;
        }
      }
      // до минута – така, както се показва
      const at = new Date(Math.round((lo + hi) / 2 / 60_000) * 60_000);
      if (sunPosition(at, lat, lon).altitude >= minAltitude) return at;
    }
    prevT = t;
    prev = cur;
  }
  return null;
}

export interface SunQiblaHint {
  /** now – точно сега (±5 мин), today – по-късно днес, tomorrow – утре. */
  kind: 'now' | 'today' | 'tomorrow';
  time: Date;
}

/** Какво да пише в картата със слънцето на екрана „Кибла“. null – картата не се показва. */
export function sunQiblaHint(now: Date, lat: number, lon: number, qibla: number): SunQiblaHint | null {
  const today = sunAtAzimuth(now, lat, lon, qibla);
  if (today) {
    const diff = today.getTime() - now.getTime();
    if (Math.abs(diff) <= SUN_NOW_MS) return { kind: 'now', time: today };
    if (diff > 0) return { kind: 'today', time: today };
  }
  const tomorrowDay = new Date(now.getFullYear(), now.getMonth(), now.getDate() + 1, 12);

```

```

const tomorrow = sunAtAzimuth(tomorrowDay, lat, lon, qibla);
return tomorrow ? { kind: 'tomorrow', time: tomorrow } : null;
}

/**
 * Истинският (астрономически) изгрев: когато горният ръб на слънцето се покаже над
 * хоризонта. Не е часът „Изгрев“ от календара на Мюфтийството – той е няколко минути
 * по-рано нарочно (предпазен интервал за края на сутрешната молитва).
 * null при полярен ден или нощ.
 */
export function sunriseTime(day: Date, lat: number, lon: number): Date | null {
  const start = new Date(day.getFullYear(), day.getMonth(), day.getDate()).getTime();
  const end = new Date(day.getFullYear(), day.getMonth(), day.getDate() + 1).getTime();
  const STEP = 10 * 60_000;
  const alt = (t: number) => sunPosition(new Date(t), lat, lon).altitude - SUN_HORIZON;
  let prevT = start;
  let prev = alt(start);
  for (let t = start + STEP; t <= end; t += STEP) {
    const cur = alt(t);
    if (prev < 0 && cur >= 0) {
      let lo = prevT;
      let hi = t;
      for (let i = 0; i < 16; i++) {
        const mid = (lo + hi) / 2;
        if (alt(mid) < 0) lo = mid;
        else hi = mid;
      }
      return new Date(Math.round(hi / 60_000) * 60_000);
    }
    prevT = t;
    prev = cur;
  }
  return null;
}

export interface SunriseBearing {
  /** Посоката на изгрева, 0-360°. */
  azimuth: number;
  /** С колко градуса Киблата е встрани от изгрева (винаги положително). */
  degrees: number;
  /** От коя страна на изгрева е Киблата, ако гледаш към него. */
  side: 'left' | 'right' | 'same';
}

/** Къде изгрява слънцето в този ден спрямо Киблата. null при полярен ден или нощ. */
export function sunriseBearing(day: Date, lat: number, lon: number, qibla: number): SunriseBearing | null {
  const sunrise = sunriseTime(day, lat, lon);
  if (!sunrise) return null;
  const { azimuth } = sunPosition(sunrise, lat, lon);
  const d = angleDelta(azimuth, qibla);
  const degrees = Math.round(Math.abs(d));
  return { azimuth, degrees, side: degrees <= 2 ? 'same' : d > 0 ? 'right' : 'left' };
}

```

Промени в `CompassDial.tsx` и `qibla.tsx` – пълният им код е в стъпка „Екранът“ (вече с тях):

- Нов prop `sun` – посоката на слънцето, когато е над хоризонта; знакът се върти заедно със скалата.
- Когато слънцето е до 3° от Киблата, вместо отделен знак около знака на Кааба светват лъчи.
- Въртящият се слой е с 12 px по-голям от компаса, за да не се отрязват лъчите.
- `useNow(30_000)` – слънцето се преизчислява на 30 секунди (то се мести с ~0,25° в минута).

- Картата със слънцето е над съвета/картите за компаса и се показва и на телефон без компас – там е особено полезна.
- Новият цвят `colors.sun` (`#F6C453`) – по-топъл от златото на приложението.

Защо Киблата не е там, където изгрива слънцето: от България Киблата е на юг-югоизток (142°), а изгревът се мести през годината – ~56° през юни, ~90° при равноденствие, ~122° през декември. Дори през декември Киблата е ~20° вдясно от изгрева.

Стъпка 7

Тестове

149 теста общо, 35 от тях нови.

qibla.test.ts src/domain/__tests__

- Посоката за София, Истанбул, Единбург, Ню Йорк и Джакарта; разстоянието София – Мека.
- Завъртане през 0°/360°, „надясно 34°“, „наляво 28°“, „обърнат“ до ±3°.
- Деклинация в София ~+5,7° и поле ~48 μT; точност по полето и в градуси.
- Посока от сензорите за 7 посоки, при наклонен телефон и при свободно падане (няма посока); изглаждане през 0°.

src/domain/__tests__/qibla.test.ts

106 реда

```
import {
  angleDelta,
  distanceToKaabaKm,
  headingFromSensors,
  magneticInfo,
  qiblaBearing,
  qualityFromAccuracy,
  qualityFromField,
  smoothHeading,
  turnInstruction,
} from '../qibla';

describe('посока към Кааба', () => {
  it.each([
    ['София', 42.6977, 23.3217, 141.9],
    ['Истанбул', 41.0082, 28.9784, 151.6],
    ['Единбург', 55.9533, -3.1883, 119.5],
    ['Ню Йорк', 40.7128, -74.006, 58.5],
    ['Джакарта', -6.2, 106.85, 295.1],
  ])('%s', (_, lat, lon, expected) => {
    expect(qiblaBearing(lat, lon)).toBeCloseTo(expected, 1);
  });

  it('София – Мека: ~2 820 км', () => {
    expect(Math.round(distanceToKaabaKm(42.6977, 23.3217))).toBeGreaterThan(2810);
    expect(Math.round(distanceToKaabaKm(42.6977, 23.3217))).toBeLessThan(2830);
  });
});

describe('накъде да се завърти', () => {
  it('най-късата посока, и през 0°/360°', () => {
    expect(angleDelta(350, 10)).toBe(20);
    expect(angleDelta(10, 350)).toBe(-20);
  });
});
```

```

    expect(angleDelta(108, 141.9)).toBeCloseTo(33.9);
  });

  it('„Завърти се надясно - 34°“ и „обърнат“ до ±3°', () => {
    expect(turnInstruction(141.9, 108)).toEqual({ degrees: 34, direction: 'right', aligned: false });
    expect(turnInstruction(141.9, 170)).toEqual({ degrees: 28, direction: 'left', aligned: false });
    expect(turnInstruction(141.9, 140).aligned).toBe(true);
    expect(turnInstruction(141.9, 144.5).aligned).toBe(true);
    expect(turnInstruction(141.9, 146).aligned).toBe(false);
  });
});

describe('магнитно поле', () => {
  it('София, 2026: деклинация ~+5,7° (изток), поле ~48 μT', () => {
    const m = magneticInfo(42.6977, 23.3217, new Date(2026, 9, 1));
    expect(m.declination).toBeGreaterThan(5);
    expect(m.declination).toBeLessThan(6.5);
    expect(m.fieldUT).toBeGreaterThan(45);
    expect(m.fieldUT).toBeLessThan(51);
  });

  it('точност по силата на полето', () => {
    expect(qualityFromField(48, 48)).toBe('high');
    expect(qualityFromField(62, 48)).toBe('medium');
    expect(qualityFromField(110, 48)).toBe('low');
    expect(qualityFromField(20, 48)).toBe('low');
  });

  it('точност на iPhone в градуси', () => {
    expect(qualityFromAccuracy(10)).toBe('high');
    expect(qualityFromAccuracy(25)).toBe('medium');
    expect(qualityFromAccuracy(45)).toBe('low');
    expect(qualityFromAccuracy(-1)).toBe('low');
  });
});

describe('посока от сензорите (оси на Android)', () => {
  // Северното полукукло: полето сочи на север и надолу (z на Земята).
  // Телефонът лежи по гръб: акселерометърът показва +g по z.
  const flat = { x: 0, y: 0, z: 1 };
  const field = (headingDeg: number) => {
    // полето в осите на телефона, когато горният край сочи headingDeg от магнитния север
    const r = (headingDeg * Math.PI) / 180;
    const north = 20; // хоризонтална съставка, μT
    return { x: -north * Math.sin(r), y: north * Math.cos(r), z: -44 };
  };

  it.each([0, 45, 90, 141.9, 180, 270, 359])('телефонът гледа на %f° - същата посока от сензорите', (h) => {
    const got = headingFromSensors(flat, field(h))!;
    expect(Math.abs(angleDelta(got, h))).toBeLessThan(0.01);
  });

  it('наклонен телефон (25° напред) - посоката остава същата', () => {
    const t = (25 * Math.PI) / 180;
    // завъртане около x: гравитацията и полето се завъртат заедно
    const rot = (v: { x: number; y: number; z: number }) => ({
      x: v.x,
      y: v.y * Math.cos(t) - v.z * Math.sin(t),
      z: v.y * Math.sin(t) + v.z * Math.cos(t),
    });
  });
});

```

```

    const got = headingFromSensors(rot(flat), rot(field(141.9)))!;
    expect(Math.abs(angleDelta(got, 141.9))).toBeLessThan(0.01);
  });

  it('без гравитация (свободно падане) - няма посока', () => {
    expect(headingFromSensors({ x: 0, y: 0, z: 0 }, field(90))).toBeNull();
  });

  it('изглаждането минава през 0°/360° по късия път', () => {
    expect(smoothHeading(355, 5, 0.5)).toBeCloseTo(0, 5);
    expect(smoothHeading(null, 123)).toBe(123);
  });
});

```

sun.test.ts src/domain/__tests__

- Положение на слънцето: София в 13:39 (~188°, ~44°), пладне на екватора, изгрев през юни/декември, нощ.
- В посоката на Киблата: София 1 окт. в 11:18, на 2 окт. в 11:17; север – никога; Ню Йорк – твърде ниско; полярна нощ.
- Картата: сутрин „днес“, в 11:20 „сега“, вечер „утре“.
- Изгрев: истинският в 07:24, на 94° – Киблата 48° вдясно; декември ~20°, юни ~86°; полярна нощ – без изгрев.

src/domain/__tests__/sun.test.ts

101 реда

```

import { qiblaBearing } from '../qibla';
import { sunAtAzimuth, sunPosition, sunQiblaHint, sunriseBearing, sunriseTime } from '../sun';

// Тестовите вървят с часова зона Europe/Sofia (jest.global-setup.js).
const SOFIA = { lat: 42.6977, lon: 23.3217 };
const QIBLA_SOFIA = qiblaBearing(SOFIA.lat, SOFIA.lon); // 141,9°
const hm = (d: Date) => `${String(d.getHours()).padStart(2, '0')}:${String(d.getMinutes()).padStart(2, '0')}`;

describe('положение на слънцето', () => {
  it('София, 1 окт. 2026, 13:39 - на юг-югозапад, високо', () => {
    const p = sunPosition(new Date('2026-10-01T13:39:00+03:00'), SOFIA.lat, SOFIA.lon);
    expect(p.azimuth).toBeGreaterThan(185);
    expect(p.azimuth).toBeLessThan(191);
    expect(p.altitude).toBeGreaterThan(42);
    expect(p.altitude).toBeLessThan(46);
  });

  it('пладне в равноденствие на екватора - почти в зенита', () => {
    // 20 март 2026, слънчево пладне на Гринуич ≈ 12:07 UTC
    const p = sunPosition(new Date('2026-03-20T12:07:00Z'), 0, 0);
    expect(p.altitude).toBeGreaterThan(88);
  });

  it('изгрев в София по сезони: юни ~56°, равноденствие ~90°, декември ~122°', () => {
    const at = (iso: string) => sunPosition(new Date(iso), SOFIA.lat, SOFIA.lon);
    expect(at('2026-06-21T05:48:00+03:00').azimuth).toBeCloseTo(56, -1);
    expect(at('2026-12-21T07:56:00+02:00').azimuth).toBeCloseTo(122, -1);
  });

  it('нощем е под хоризонта', () => {
    expect(sunPosition(new Date('2026-10-01T23:00:00+03:00'), SOFIA.lat,

```

```

SOFIA.lon).altitude).toBeLessThan(-30);
});
});

describe('слънцето в посоката на Киблата', () => {
  it('София, 1 окт. 2026 - в 11:18, на ~37° височина', () => {
    const at = sunAtAzimuth(new Date(2026, 9, 1), SOFIA.lat, SOFIA.lon, QIBLA_SOFIA!);
    expect(hm(at)).toBe('11:18');
    expect(sunPosition(at, SOFIA.lat, SOFIA.lon).altitude).toBeCloseTo(37, 0);
  });

  it('утре минута по-рано', () => {
    expect(hm(sunAtAzimuth(new Date(2026, 9, 2), SOFIA.lat, SOFIA.lon, QIBLA_SOFIA!))).toBe('11:17');
  });

  it('посока, в която слънцето никога не застава (север) - null', () => {
    expect(sunAtAzimuth(new Date(2026, 9, 1), SOFIA.lat, SOFIA.lon, 0)).toBeNull();
  });

  it('твърде ниско над хоризонта - null', () => {
    // Ню Йорк: Киблата е на ~58°; през октомври слънцето е там само под хоризонта
    const ny = { lat: 40.7128, lon: -74.006 };
    expect(sunAtAzimuth(new Date(2026, 9, 1), ny.lat, ny.lon, qiblaBearing(ny.lat, ny.lon))).toBeNull();
  });

  it('полярна нощ - null', () => {
    expect(sunAtAzimuth(new Date(2026, 11, 21), 78.22, 15.65, qiblaBearing(78.22, 15.65))).toBeNull();
  });
});

describe('текстът в картата', () => {
  const hint = (iso: string) => sunQiblaHint(new Date(iso), SOFIA.lat, SOFIA.lon, QIBLA_SOFIA!);

  it('сутрин - „днес в 11:18“', () => {
    const h = hint('2026-10-01T09:30:00+03:00');
    expect(h.kind).toBe('today');
    expect(hm(h.time)).toBe('11:18');
  });

  it('в 11:20 - „сега“', () => {
    expect(hint('2026-10-01T11:20:00+03:00').kind).toBe('now');
  });

  it('вечер - „утре в 11:17“', () => {
    const h = hint('2026-10-01T20:30:00+03:00');
    expect(h.kind).toBe('tomorrow');
    expect(hm(h.time)).toBe('11:17');
  });
});

describe('изгревът спрямо Киблата', () => {
  it('истинският изгрев в София на 1 окт. 2026 е в 07:24 (Мюфтийството дава 07:16 - с предпазен интервал)', () => {
    expect(hm(sunriseTime(new Date(2026, 9, 1), SOFIA.lat, SOFIA.lon!))).toBe('07:24');
  });

  it('София, 1 окт.: изгрев на ~94°, Киблата е ~48° вдясно', () => {
    const b = sunriseBearing(new Date(2026, 9, 1), SOFIA.lat, SOFIA.lon, QIBLA_SOFIA!);
    expect(Math.round(b.azimuth)).toBe(94);
    expect(b.side).toBe('right');
    expect(b.degrees).toBe(48);
  });
});

```

```
});

it('декември: изгрев на ~122° – Киблата е ~20° вдясно; юни: ~56° – ~86° вдясно', () => {
  expect(sunriseBearing(new Date(2026, 11, 21), SOFIA.lat, SOFIA.lon,
    QIBLA_SOFIA).degrees).toBeCloseTo(20, -1);
  expect(sunriseBearing(new Date(2026, 5, 21), SOFIA.lat, SOFIA.lon,
    QIBLA_SOFIA).degrees).toBeCloseTo(86, -1);
});

it('полярна нощ – няма изгрев', () => {
  expect(sunriseBearing(new Date(2026, 11, 21), 78.22, 15.65, qiblaBearing(78.22, 15.65))).toBeNull();
});
});
```

Трябва да видиш: Tests: 149 passed, 149 total

npm test

Стъпка 8

Проверка и край на етапа

Етап 7 е готов, когато на телефона:

- Табът „Кибла“ показва компаса; скалата се върти плавно, „С“ (в оранжево) сочи север.
- Стрелката сочи Киблата: свери с джамия или с друго приложение за Кибла – разлика до ~5° е нормална за компас на телефон.
- Завърти се към Кибла: лентата светва в злато, „✓ Обърнат си към Кибла“, кратка вибрация.
- Сложи телефона до метал или магнит: точността пада на „ниска“ и се появява „Калибрирай компаса“. Махни го и направи осмица – точността се връща.
- Градусите и километрите се сменят при друго място (Настройки → Смени мястото).
- Слънцето: ✱ на компаса е там, където го виждаш в небето. В часа от картата („Днес в 11:18...“) застани с лице към слънцето – стрелката трябва да сочи право напред.

Съвет за проверка: дръж телефона хоризонтално, на нивото на кръста, далеч от лаптоп, кола или метална маса.

Терминал

```
git add .
git commit -m "Етап 7: Кибла"
git tag stage-7
```

След това – етап 5 (пълният езан на Android, Huawei без Google, методът според държавата).